IT4931 / IT5384 / IT4043E · SOICT · HUST

Hadoop Distributed File System

Lecture 3 — HDFS

Architecture · blocks · replication · read/write · formats · modern storage

Learning outcomes (CLO2, CLO3)

1. **Explain** HDFS's role as the **storage layer** (throughput, scale, fault tolerance)
2. **Describe** NameNode, DataNode, Secondary NameNode / HA idea
3. **Reason about** blocks, replication, and **rack awareness**
4. **Sketch** read and write pipelines (client ↔ NN ↔ DNs)
5. **Compare** common storage formats (text, SequenceFile, Parquet/ORC)
6. **Connect** HDFS ideas to object storage / lakehouse (preview)

Roadmap today

1. **3.1** Introduction — role & design goals
2. **3.2** Architecture — NN / DN / blocks / rack awareness
3. **3.3** Read, write, organize — paths & commands
4. **3.4** Storage formats — Avro · Parquet · ORC
5. **Bridge** — HDFS → modern platforms → later lectures

3.1 Introduction to HDFS

Role of HDFS

- **Storage layer** of classic Hadoop platforms
- Optimized for **large files**, sequential / streaming reads
- Design goals: **scale-out**, **fault tolerance**, **high throughput** (not low latency random I/O)
- Pattern: **write once (append), read many**

From L02 overview → **depth here**.

Local FS vs distributed FS (context)

	Local FS (e.g. NTFS/ext4)	Distributed FS (HDFS-class)
Namespace	One machine	Cluster-wide paths
Capacity	Disks in one box	Sum of many nodes
Failure	Backup / RAID locally	**Replication** across nodes
Client view	Mount one volume	Unified tree; location hidden

HDFS = distributed FS optimized for **large sequential** analytics workloads.

Transparency & availability (goals)

Goal	Meaning in HDFS-class systems
Location transparency	Path does not encode physical machine
Availability	Failures shouldn't stop access → **replication**
Unified namespace	Hierarchical paths like `/data/logs/…`

HDFS realizes these with NameNode metadata + replicated blocks.

3.2 HDFS architecture

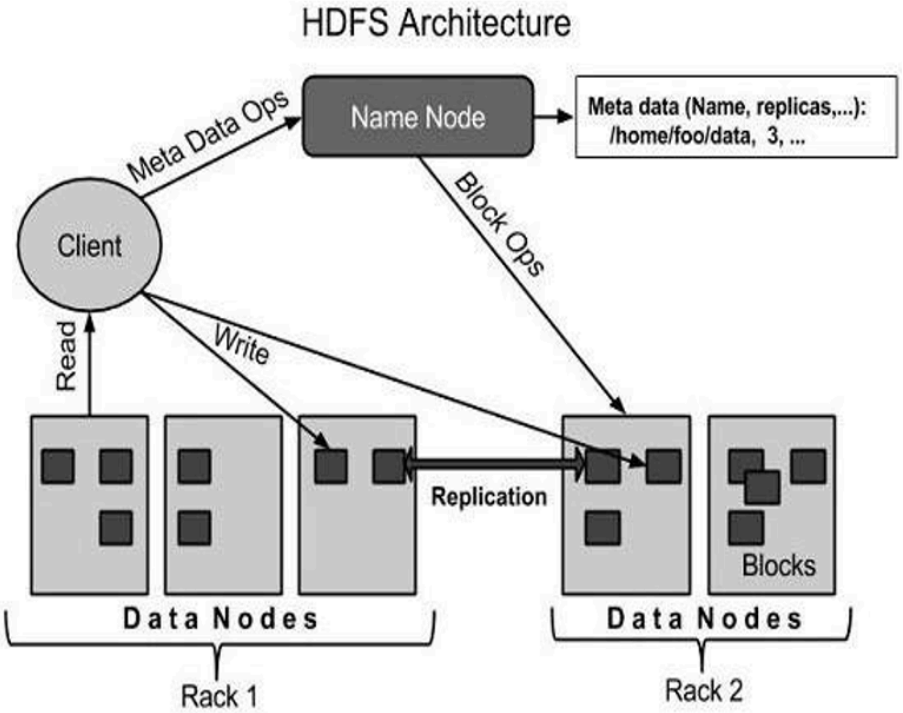
Master / worker roles

Component	Role
NameNode	Namespace + block map (metadata in RAM)
DataNode	Stores blocks on local disk; heartbeats + block reports
Secondary NameNode	Checkpoint helper (merge FsImage + EditLog) — **not** a hot standby in classic setups

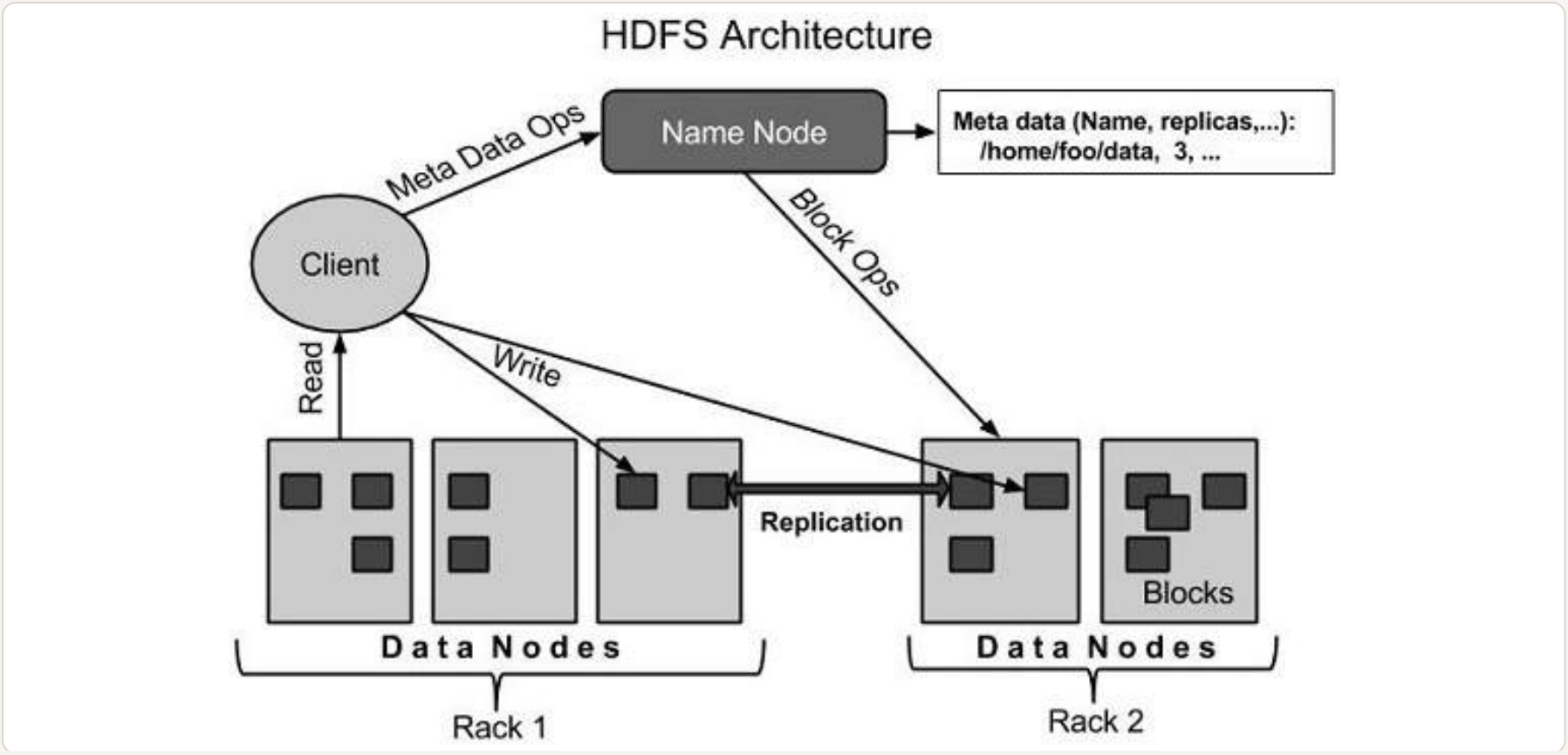
Modern deployments often use **NameNode HA** (Active/Standby) instead of relying on Secondary NN alone.

HDFS architecture

Client asks NameNode for metadata, then **streams data directly** with DataNodes.



HDFS architecture (alternate view)



NameNode — the librarian

- Holds **namespace** (dirs/files), permissions, times, sizes
- Maps file → **block IDs** → DataNode locations
- Metadata in **RAM** ⇒ fast, but classic SPOF risk / heap limits
- Does **not** sit on the data path for bulk transfer

DataNode — the shelves

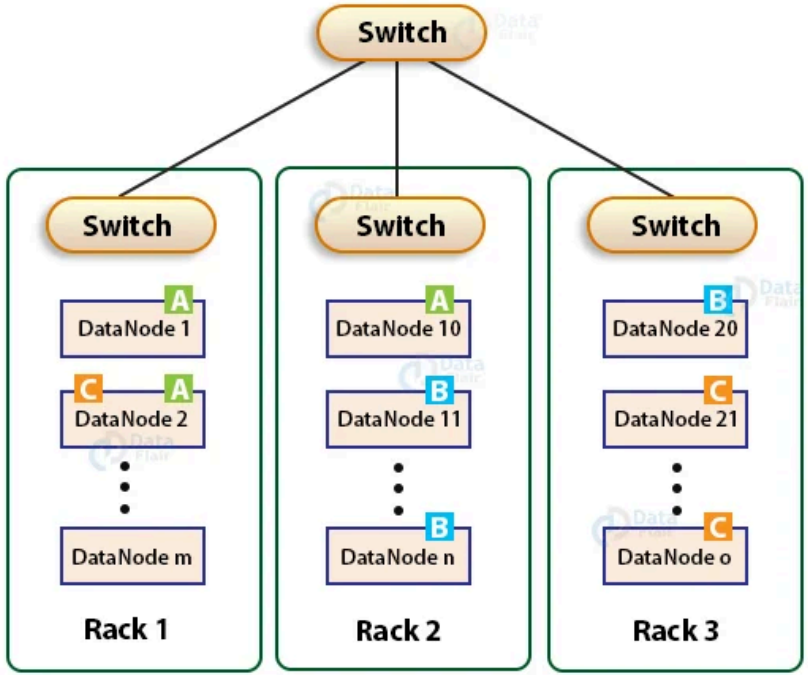
- Stores block files on local disks
- Serves client read/write of block bytes
- Periodic **heartbeat** + **block report** to NameNode
- Missing heartbeat ⇒ NN marks dead → **re-replication**

Blocks

- Default block size historically **128 MB** (configurable; was 64 MB)
- Large blocks ⇒ fewer seeks, less NN metadata, good sequential throughput
- Small file still uses only its actual bytes on disk, but still costs NN metadata entries

Replication & rack awareness

Default replication factor **3** with rack-aware placement for failure domains.



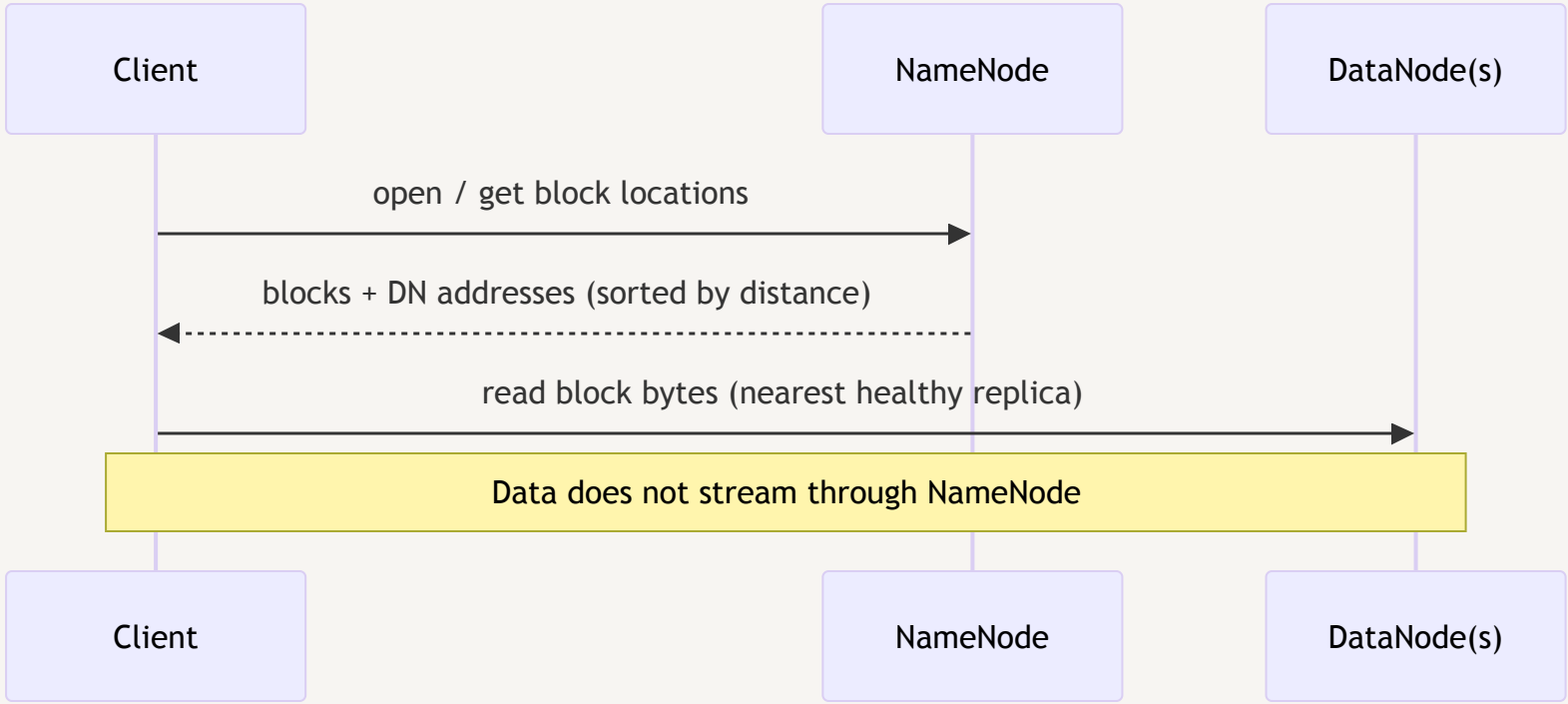
Placement policy (typical)

1. One replica on a node in the writer's rack (when applicable)
2. Second replica on a **different rack**
3. Third on another node in that second rack

⇒ Survive node failure **and** whole-rack failure more often than random placement.

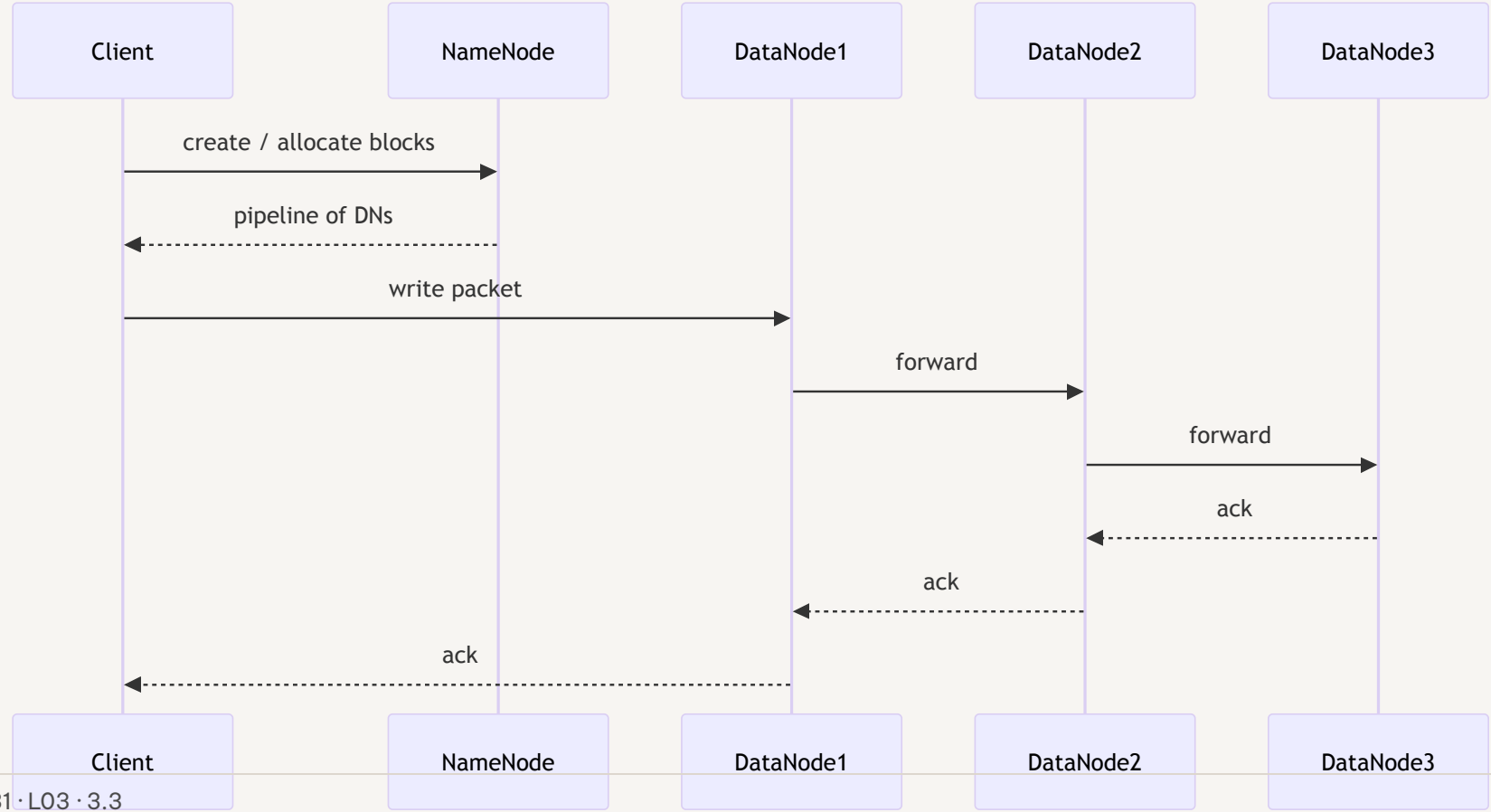
3.3 Read, write, organize

Read path



If a DN fails mid-read, client retries another replica.

Write path (pipeline)



Organize files — mental model

- Hierarchical paths: `/user/alice/logs/2026-09-04/`
- Directories + permissions (Unix-inspired)
- Prefer **fewer large files** over millions of tiny files (NN metadata pressure)

Common CLI (sketch)

```
hdfs dfs -ls /data
hdfs dfs -mkdir -p /data/raw
hdfs dfs -put local.csv /data/raw/
hdfs dfs -get /data/raw/local.csv ./
hdfs dfs -cat /data/raw/local.csv | head
hdfs dfs -rm -r /data/tmp
```

Same ideas appear in cloud CLIs (`aws s3 cp` , `gsutil`) with different consistency models.

3.4 Data storage formats

Why formats matter on HDFS / object stores

HDFS stores **opaque byte files**. Engines (MapReduce, Spark, Hive, Trino) decide *how* to parse them.

Concern	If you get it wrong…
Splittability	Cannot parallelize a large file → one fat task
Compression	Tiny files or non-splittable gzip → slow / skewed jobs
Row vs columnar	Analytics reads 50 columns to use 2 → I/O explosion
Schema / types	Silent parse errors; hard evolution across producers

Format choice is a storage-layer decision with compute consequences.

Landscape of common formats

Family	Examples	Shape
Line / text	CSV, TSV, JSON Lines, logs	Human-readable; expensive at PB scale
Hadoop containers	SequenceFile, RCFile (legacy)	Binary; MapReduce-era
Row + schema	**Avro**	Whole records; great for streams / ETL
Columnar	**Parquet**, **ORC**	Column pages + stats; analytics default

Today's lake path: events often **Avro/JSON** in flight → land as **Parquet** (sometimes ORC) on disk.

Text / CSV / JSON — when (not) to use

Pros	Cons at scale
Easy to eyeball / debug	Parsing CPU heavy; weak typing
Universal tools	CSV dialect hell; nested JSON ≠ flat tables
Fine for configs, samples	Compression often **non-splittable** (gzip)

Use for: small extracts, debugging, config.
Avoid as the **system of record** for warehouse-scale scans — prefer Avro/Parquet.

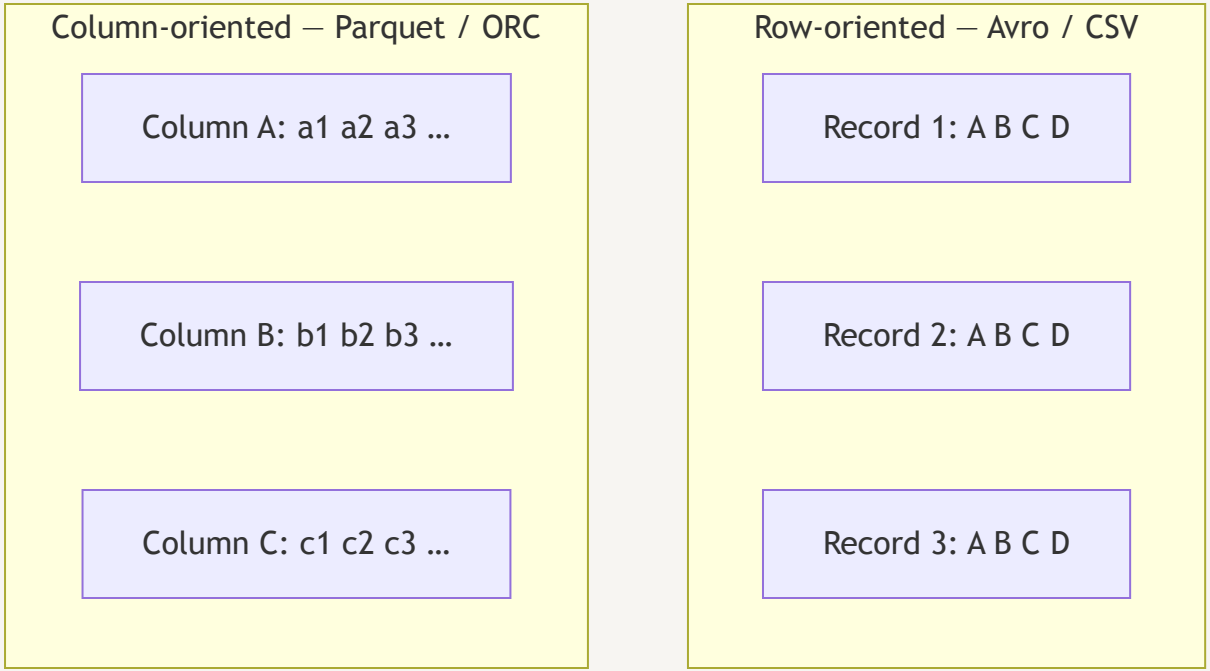
Avro — row format with a schema

- **Row-oriented** binary encoding + **JSON schema** (writer/reader schemas)
- Strong at **schema evolution** (add optional fields, rename with aliases)
- Natural fit for **Kafka / Flume / NiFi** payloads and ETL staging
- Splittable with Object Container File; compress with Snappy/Deflate/…

```
{ "type": "record", "name": "Click",
  "fields": [
    { "name": "user_id", "type": "long" },
    { "name": "ts", "type": "long" },
    { "name": "url", "type": "string" }
  ] }
```

Think: “typed SequenceFile for the streaming era.”

Row vs columnar — picture



Analytics query `SELECT B, C WHERE A > …` touches **only needed columns** in columnar layouts.

Row vs columnar — trade-offs

	Row (Avro, JSON lines)	Columnar (Parquet, ORC)
Point lookup / whole event	Natural	Reconstruct from columns
Scan 2 of 100 columns	Read almost everything	Read ~2 columns + stats
Compression	Good	Excellent (typed runs, dict)
Write pattern	Append records easily	Buffer row groups / stripes

Parquet — analytics workhorse

- **Columnar** file format (Twitter/Cloudera lineage; now Apache)
- File = **row groups** → column chunks → pages (+ optional dictionary)
- Embedded **min/max / null counts** → predicate pushdown / skip row groups
- First-class in **Spark, Hive, Presto/Trino, DuckDB, pandas**
- Default **data file** under Iceberg / Delta / Hudi tables (L04)

*When students say "put it in the lake," they usually mean **Parquet objects** under a table layout.*

ORC — Hive's columnar sibling

	Parquet	ORC
Origin / ecosystem	Broad (Spark, Trino, lakes)	Hive / Hadoop SQL heavy
Layout idea	Row groups + column pages	**Stripes** + indexes / bloom
Compression	Snappy, ZSTD, …	Often very strong defaults
Interop	Widest today	Excellent inside Hive stack

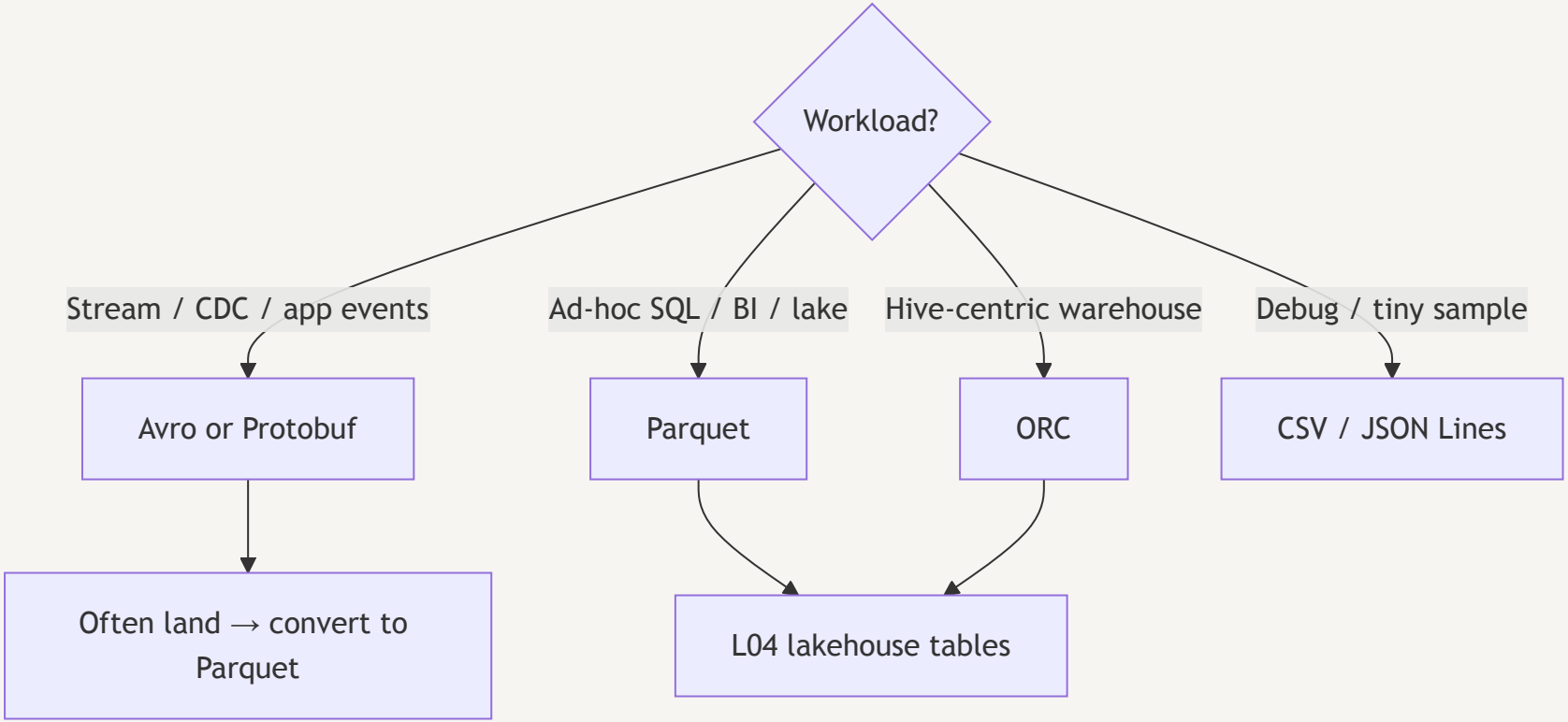
Same **columnar** idea — pick by **engine ecosystem**, not "which is smarter."

Splittability & compression (practical)

Codec / setup	Splittable on HDFS?	Note
Plain / Snappy block	Yes (with container)	Common with Parquet/Avro
gzip whole file	**No**	One mapper/task — avoid for big files
bzip2	Yes (slow)	Rare now
Many tiny files	“Parallel” but NN/S3 listing pain	Prefer large files (~128MB–1GB+)

Design rule: large, splittable, columnar files beat millions of tiny gzipped JSON.

Choosing a format — decision sketch



Bridge to modern storage

From HDFS to object storage

Classic	Modern common pattern
HDFS blocks + NN	S3 / ADLS / GCS objects
Replication in FS	Cloud durability classes / erasure coding
`hdfs dfs`	Cloud SDKs / `s3a://` connectors
Same need	Partitioning, large files, columnar formats

Principles transfer; **control plane** differs (no NameNode, different consistency).

Where this leads in the course

Later	Connection
L04 Lakehouse	Tables **on** these files (often Parquet) + ACID/time travel
L05 NoSQL	Serving / alternative models when FS tables are not enough
L08 Kafka	Streams often carry **Avro**/JSON payloads into the lake
L10–L11 Spark	Engines that read Parquet/ORC from HDFS or object storage

Takeaways

1. HDFS = **distributed, block-based**, write-once/read-many storage
2. **NameNode** metadata · **DataNodes** bytes · client data path bypasses NN
3. **Large blocks + replication + rack awareness** = throughput & durability
4. Know **read vs write** pipelines at a sketch level
5. Formats: **Avro** (streams) · **Parquet/ORC** (analytics) · mind **splittability**
6. Object storage / lakehouse reuse the same storage-layer thinking

Next

L04 — Lakehouse